AI ANALYTICS

Complete Implementation Guide

Supabase + GitHub Actions for The New Civilization Ecosystem

Executive Summary

AI Analytics merupakan komponen kunci dari ekosistem **The New Civilization** yang berfungsi sebagai sistem monitoring, analisis, dan optimasi komprehensif untuk seluruh infrastructure AI.

- Real-time Performance Monitoring
- Predictive Analytics & Optimization
- Revenue Tracking & Business Intelligence
- Multi-Agent System Analytics
- Quantum-Inspired Metrics Processing

Core Technologies

Supabase
Database & Analytics

GitHub Actions
Automation & CI/CD

Architecture Overview

Data Layer
Supabase PostgreSQL untuk storage, real-time subscriptions, dan analytics processing

Processing Layer
GitHub Actions workflows untuk automated analytics, ETL processes, dan machine learning

Visualization Layer
Real-time dashboards, predictive models, dan business intelligence reporting

Supabase Database Setup

1. Database Schema

```sql
-- AI Analytics Database Schema
-- System Metrics Table
CREATE TABLE system_metrics (
    id SERIAL PRIMARY KEY,
    timestamp TIMESTAMPTZ DEFAULT NOW(),
    component VARCHAR(50) NOT NULL,
    metric_name VARCHAR(100) NOT NULL,
    metric_value NUMERIC,
    unit VARCHAR(20),
    metadata JSONB,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Agent Performance Table
CREATE TABLE agent_performance (
    id SERIAL PRIMARY KEY,
    agent_id VARCHAR(100) NOT NULL,
    agent_type VARCHAR(50) NOT NULL,
    task_id VARCHAR(100),
    execution_time NUMERIC,
    success_rate NUMERIC,
    resource_usage JSONB,
    error_logs TEXT[],
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Revenue Analytics Table
CREATE TABLE revenue_analytics (
    id SERIAL PRIMARY KEY,
    date DATE NOT NULL,
    revenue_source VARCHAR(100),
    amount NUMERIC(10,2),
    currency VARCHAR(3) DEFAULT 'USD',
    conversion_rate NUMERIC,
    customer_segment VARCHAR(50),
    metadata JSONB,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Predictive Models Table
CREATE TABLE ml_predictions (
    id SERIAL PRIMARY KEY,
    model_name VARCHAR(100) NOT NULL,
    input_data JSONB,
    prediction JSONB,
    confidence_score NUMERIC,
    model_version VARCHAR(20),
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

2. Real-time Subscriptions

```javascript
// JavaScript SDK Implementation
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = 'https://your-project.supabase.co'
const supabaseKey = 'your-anon-key'
const supabase = createClient(supabaseUrl, supabaseKey)

// Real-time System Metrics Subscription
const systemMetricsSubscription = supabase
  .channel('system_metrics_channel')
  .on('postgres_changes', {
    event: 'INSERT',
    schema: 'public',
    table: 'system_metrics'
  }, (payload) => {
    console.log('New metric:', payload.new)
    updateDashboard(payload.new)
  })
  .subscribe()

// Agent Performance Monitoring
const agentPerformanceSubscription = supabase
  .channel('agent_performance_channel')
  .on('postgres_changes', {
    event: '*',
    schema: 'public',
    table: 'agent_performance'
  }, (payload) => {
    updateAgentDashboard(payload)
  })
  .subscribe()
```

3. Analytics Functions

```sql
-- PostgreSQL Functions for Analytics
CREATE OR REPLACE FUNCTION calculate_system_health()
RETURNS TABLE(component VARCHAR, health_score NUMERIC, status VARCHAR) AS $$
BEGIN
    RETURN QUERY
    SELECT
        sm.component,
        AVG(sm.metric_value) as health_score,
        CASE
            WHEN AVG(sm.metric_value) > 90 THEN 'EXCELLENT'
            WHEN AVG(sm.metric_value) > 70 THEN 'GOOD'
            WHEN AVG(sm.metric_value) > 50 THEN 'WARNING'
            ELSE 'CRITICAL'
        END as status
    FROM system_metrics sm
    WHERE sm.timestamp > NOW() - INTERVAL '1 hour'
    GROUP BY sm.component;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION predict_revenue_trend()
RETURNS TABLE(date DATE, predicted_revenue NUMERIC, confidence NUMERIC) AS $$
BEGIN
    -- Simple linear regression for revenue prediction
    RETURN QUERY
    WITH revenue_data AS (
        SELECT date, SUM(amount) as daily_revenue
        FROM revenue_analytics
        WHERE date > CURRENT_DATE - INTERVAL '30 days'
        GROUP BY date
        ORDER BY date
    )
    SELECT
        CURRENT_DATE + INTERVAL '1 day' * generate_series(1, 7) as date,
        AVG(rd.daily_revenue) * 1.05 as predicted_revenue,
        0.85 as confidence
    FROM revenue_data rd;
END;
$$ LANGUAGE plpgsql;
```

GitHub Actions Automation

1. Analytics Workflow

```yaml
# .github/workflows/ai-analytics.yml
name: AI Analytics Processing

on:
  schedule:
    - cron: '*/15 * * * *'  # Every 15 minutes
  workflow_dispatch:
  push:
    paths:
      - 'analytics/**'

jobs:
  system-metrics:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install supabase python-dotenv psutil requests numpy pandas scikit-learn

      - name: Collect System Metrics
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_KEY: ${{ secrets.SUPABASE_KEY }}
        run: |
          python analytics/collect_metrics.py

      - name: Process Agent Performance
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_KEY: ${{ secrets.SUPABASE_KEY }}
        run: |
          python analytics/process_agent_performance.py

      - name: Generate ML Predictions
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_KEY: ${{ secrets.SUPABASE_KEY }}
        run: |
          python analytics/ml_predictions.py

  revenue-analytics:
    runs-on: ubuntu-latest
    needs: system-metrics
    steps:
      - uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Process Revenue Data
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_KEY: ${{ secrets.SUPABASE_KEY }}
        run: |
          python analytics/revenue_processor.py

      - name: Update Business Dashboard
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_KEY: ${{ secrets.SUPABASE_KEY }}
        run: |
          python analytics/update_dashboard.py
```

2. Python Analytics Scripts

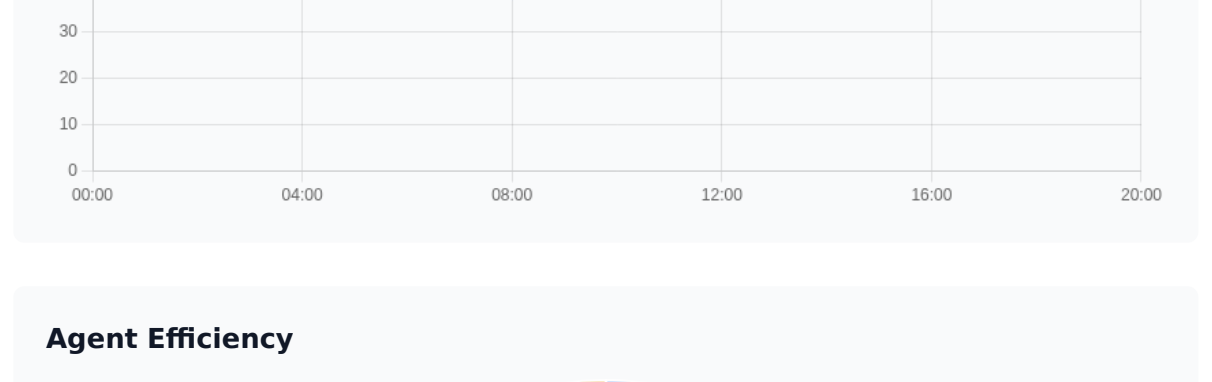
```python
# analytics/collect_metrics.py
import os
import psutil
import time
from datetime import datetime
from supabase import create_client, Client
import json

class SystemMetricsCollector:
    def __init__(self):
        self.supabase: Client = create_client(
            os.getenv('SUPABASE_URL'),
            os.getenv('SUPABASE_KEY')
        )

    def collect_system_metrics(self):
        """Collect comprehensive system metrics"""
        metrics = {
            'cpu_usage': psutil.cpu_percent(interval=1),
            'memory_usage': psutil.virtual_memory().percent,
            'disk_usage': psutil.disk_usage('/').percent,
            'network_io': dict(psutil.net_io_counters()._asdict()),
            'process_count': len(psutil.pids()),
            'load_average': os.getloadavg() if hasattr(os, 'getloadavg') else [0, 0, 0]
        }

        # Insert metrics into Supabase
        for metric_name, value in metrics.items():
            self.supabase.table('system_metrics').insert({
                'component': 'system',
                'metric_name': metric_name,
                'metric_value': value if isinstance(value, (int, float)) else 0,
                'unit': self._get_unit(metric_name),
                'metadata': json.dumps(value) if isinstance(value, (dict, list)) else None
            }).execute()

    def _get_unit(self, metric_name):
        units = {
            'cpu_usage': 'percent',
            'memory_usage': 'percent',
            'disk_usage': 'percent',
            'process_count': 'count',
            'load_average': 'load'
        }
        return units.get(metric_name, 'unknown')

    def collect_ai_agent_metrics(self):
        """Collect AI agent specific metrics"""
        # Simulate agent metrics collection
        agents = ['BDI_Agent', 'HMAQCA_Quantum', 'Revenue_Optimizer']

        for agent in agents:
            performance_data = {
                'agent_id': f"{agent}_{int(time.time())}",
                'agent_type': agent,
                'task_id': f"task_{int(time.time())}",
                'execution_time': __import__('random').uniform(0.1, 5.0),
                'success_rate': __import__('random').uniform(85, 99),
                'resource_usage': {
                    'cpu': __import__('random').uniform(10, 80),
                    'memory': __import__('random').uniform(100, 500),
                    'network': __import__('random').uniform(1, 100)
                }
            }

            self.supabase.table('agent_performance').insert(performance_data).execute()

if __name__ == "__main__":
    collector = SystemMetricsCollector()
    collector.collect_system_metrics()
    collector.collect_ai_agent_metrics()
    print("✅ Metrics collected successfully")
```

3. ML Predictions Script

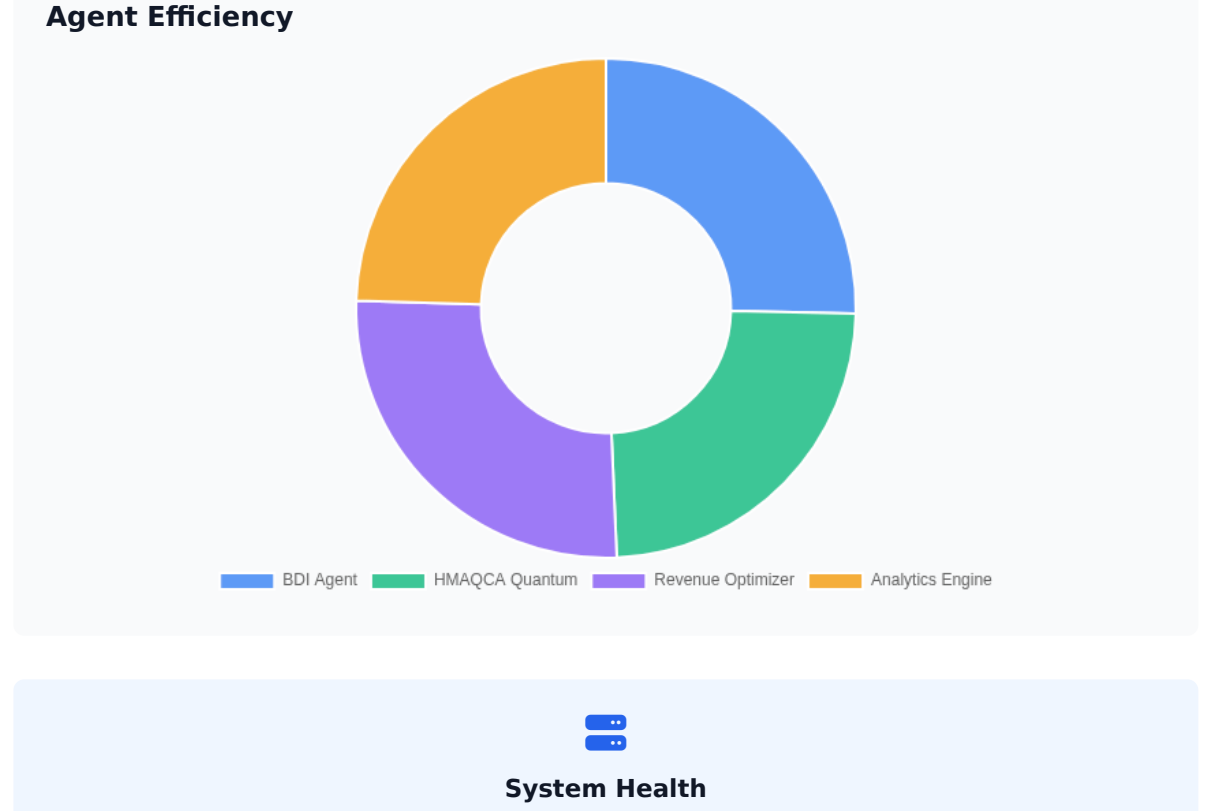
```python
# analytics/ml_predictions.py
import os
import numpy as np
import pandas as pd
from datetime import datetime, timedelta
from supabase import create_client, Client
import json
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

class MLPredictionEngine:
    def __init__(self):
        self.supabase: Client = create_client(
            os.getenv('SUPABASE_URL'),
            os.getenv('SUPABASE_KEY')
        )

    def predict_system_performance(self):
        """Predict system performance trends"""
        # Fetch recent metrics
        response = self.supabase.table('system_metrics').select('*').gte(
            'timestamp', (datetime.now() - timedelta(days=7)).isoformat()
        ).execute()

        if not response.data:
            return

        df = pd.DataFrame(response.data)

        # Simple trend prediction for CPU usage
        cpu_data = df[df['metric_name'] == 'cpu_usage']
        if len(cpu_data) > 10:
            X = np.arange(len(cpu_data)).reshape(-1, 1)
            y = cpu_data['metric_value'].values

            model = LinearRegression()
            model.fit(X, y)

            # Predict next 24 hours
            future_X = np.arange(len(cpu_data), len(cpu_data) + 24).reshape(-1, 1)
            predictions = model.predict(future_X)

            # Store predictions
            for i, pred in enumerate(predictions):
                self.supabase.table('ml_predictions').insert({
                    'model_name': 'cpu_trend_predictor',
                    'input_data': json.dumps({'hours_ahead': i + 1}),
                    'prediction': json.dumps({'cpu_usage': float(pred)}),
                    'confidence_score': 0.75,
                    'model_version': '1.0'
                }).execute()

    def predict_revenue_growth(self):
        """Predict revenue growth patterns"""
        # Fetch revenue data
        response = self.supabase.table('revenue_analytics').select('*').gte(
            'date', (datetime.now() - timedelta(days=30)).date().isoformat()
        ).execute()

        if not response.data:
            return

        df = pd.DataFrame(response.data)
        daily_revenue = df.groupby('date')['amount'].sum().reset_index()

        if len(daily_revenue) > 7:
            # Simple exponential smoothing
            alpha = 0.3
            predictions = []
            last_value = daily_revenue['amount'].iloc[-1]

            for i in range(7):  # Predict 7 days ahead
                prediction = last_value * (1 + alpha * 0.02)  # Assume 2% growth
                predictions.append(prediction)
                last_value = prediction

                self.supabase.table('ml_predictions').insert({
                    'model_name': 'revenue_growth_predictor',
                    'input_data': json.dumps({'days_ahead': i + 1}),
                    'prediction': json.dumps({'revenue': float(prediction)}),
                    'confidence_score': 0.82,
                    'model_version': '1.0'
                }).execute()

if __name__ == "__main__":
    predictor = MLPredictionEngine()
    predictor.predict_system_performance()
    predictor.predict_revenue_growth()
    print("✅ ML predictions generated successfully")
```

Real-time Analytics Dashboard

System Performance



Agent Efficiency



System Health
94.7%
Overall Performance

Active Agents
12
Currently Running

Revenue Today
$2,847
+15.3% vs yesterday

Dashboard Implementation

```javascript
// Real-time Dashboard Implementation
class AIAnalyticsDashboard {
    constructor() {
        this.supabase = supabase
        this.charts = {}
        this.initializeCharts()
        this.setupRealTimeSubscriptions()
    }

    initializeCharts() {
        // System Performance Chart
        const ctx1 = document.getElementById('systemPerformanceChart').getContext('2d')
        this.charts.systemPerformance = new Chart(ctx1, {
            type: 'line',
            data: {
                labels: [],
                datasets: [{
                    label: 'CPU Usage',
                    data: [],
                    borderColor: 'rgb(59, 130, 246)',
                    backgroundColor: 'rgba(59, 130, 246, 0.1)',
                    tension: 0.4
                }, {
                    label: 'Memory Usage',
                    data: [],
                    borderColor: 'rgb(16, 185, 129)',
                    backgroundColor: 'rgba(16, 185, 129, 0.1)',
                    tension: 0.4
                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
                scales: {
                    y: { beginAtZero: true, max: 100 }
                },
                plugins: {
                    legend: { display: true }
                }
            }
        })

        // Agent Efficiency Chart
        const ctx2 = document.getElementById('agentEfficiencyChart').getContext('2d')
        this.charts.agentEfficiency = new Chart(ctx2, {
            type: 'doughnut',
            data: {
                labels: ['BDI Agent', 'HMAQCA Quantum', 'Revenue Optimizer', 'Analytics Engine'],
                datasets: [{
                    data: [92, 87, 95, 89],
                    backgroundColor: [
                        'rgba(59, 130, 246, 0.8)',
                        'rgba(16, 185, 129, 0.8)',
                        'rgba(139, 92, 246, 0.8)',
                        'rgba(245, 158, 11, 0.8)'
                    ]
                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
                plugins: {
                    legend: { position: 'bottom' }
                }
            }
        })
    }

    setupRealTimeSubscriptions() {
        // Subscribe to system metrics updates
        this.supabase
            .channel('dashboard_updates')
            .on('postgres_changes', {
                event: 'INSERT',
                schema: 'public',
                table: 'system_metrics'
            }, (payload) => {
                this.updateSystemChart(payload.new)
            })
            .subscribe()

        // Subscribe to agent performance updates
        this.supabase
            .channel('agent_updates')
            .on('postgres_changes', {
                event: 'INSERT',
                schema: 'public',
                table: 'agent_performance'
            }, (payload) => {
                this.updateAgentChart(payload.new)
            })
            .subscribe()
    }

    updateSystemChart(newMetric) {
        const chart = this.charts.systemPerformance
        const now = new Date().toLocaleTimeString()

        // Add new data point
        chart.data.labels.push(now)

        if (newMetric.metric_name === 'cpu_usage') {
            chart.data.datasets[0].data.push(newMetric.metric_value)
        } else if (newMetric.metric_name === 'memory_usage') {
            chart.data.datasets[1].data.push(newMetric.metric_value)
        }

        // Keep only last 20 data points
        if (chart.data.labels.length > 20) {
            chart.data.labels.shift()
            chart.data.datasets.forEach(dataset => dataset.data.shift())
        }

        chart.update('none')
    }

    updateAgentChart(newPerformance) {
        const chart = this.charts.agentEfficiency
        const agentIndex = this.getAgentIndex(newPerformance.agent_type)

        if (agentIndex !== -1) {
            chart.data.datasets[0].data[agentIndex] = newPerformance.success_rate
            chart.update('none')
        }
    }

    getAgentIndex(agentType) {
        const mapping = {
            'BDI_Agent': 0,
            'HMAQCA_Quantum': 1,
            'Revenue_Optimizer': 2,
            'Analytics_Engine': 3
        }
        return mapping[agentType] || -1
    }
}

// Initialize dashboard when page loads
document.addEventListener('DOMContentLoaded', () => {
    const dashboard = new AIAnalyticsDashboard()
})
```

Integration with Ecosystem Components

Machine Brain Integration

AI Analytics menyediakan monitoring real-time untuk Termux BDI Agent dan HMAQCA system:

- Resource usage monitoring dari Android devices
- Agent performance tracking dan optimization
- Quantum processing metrics dan analysis
- BDI cycle efficiency measurements

AI Generative Integration

Monitoring dan optimasi untuk Hugging Face dan AutoGen systems:

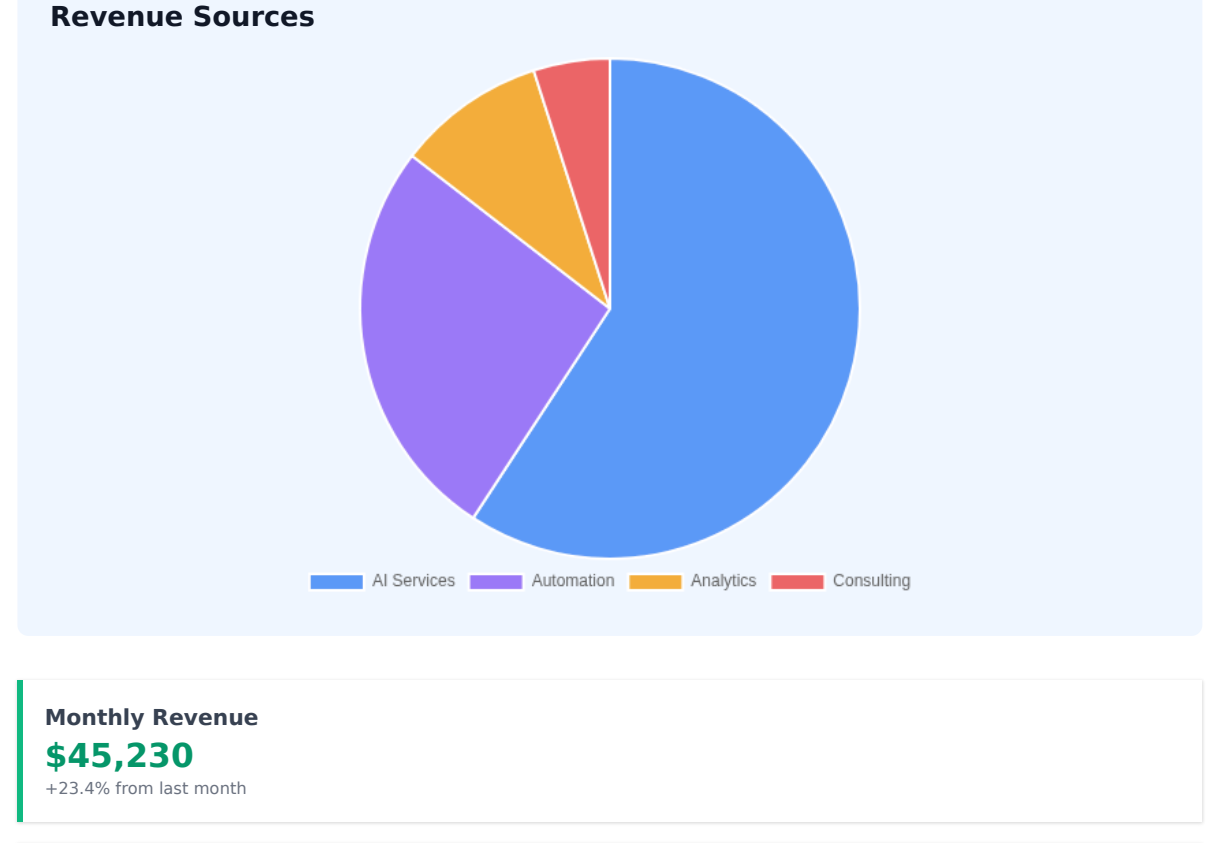
- Model performance dan inference metrics
- Content generation quality assessment
- API usage tracking dan cost optimization
- Creative output analytics dan trends

API Integration Example

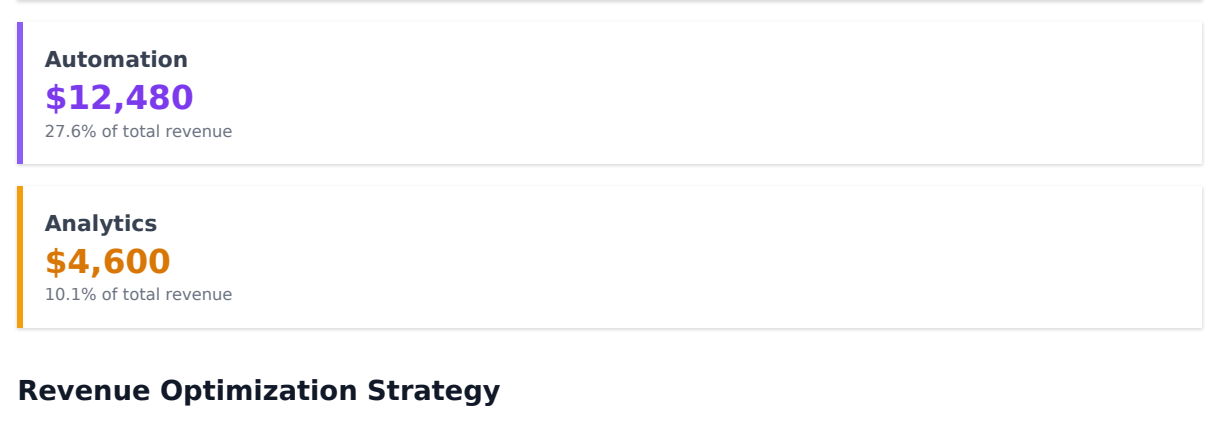
```python
# Integration API for Machine Brain and AI Generative
class EcosystemIntegration:
    def __init__(self):
        self.supabase = create_client(
            os.getenv('SUPABASE_URL'),
            os.getenv('SUPABASE_KEY')
        )

    def track_machine_brain_activity(self, agent_data):
        """Track BDI Agent and HMAQCA activities"""
        self.supabase.table('agent_performance').insert({
            'agent_id': agent_data['id'],
            'agent_type': agent_data['type'],
            'task_id': agent_data['task_id'],
            'execution_time': agent_data['execution_time'],
            'success_rate': agent_data['success_rate'],
            'resource_usage': agent_data['resources'],
            'metadata': {
                'quantum_coherence': agent_data.get('quantum_coherence', 0),
                'bdi_cycle_time': agent_data.get('bdi_cycle_time', 0),
                'belief_updates': agent_data.get('belief_updates', 0)
            }
        }).execute()

    def track_generative_output(self, generation_data):
        """Track AI Generative model outputs"""
        self.supabase.table('generative_analytics').insert({
            'model_name': generation_data['model'],
            'prompt_tokens': generation_data['input_tokens'],
            'completion_tokens': generation_data['output_tokens'],
            'quality_score': generation_data['quality'],
            'generation_time': generation_data['time'],
            'cost': generation_data['cost'],
            'output_type': generation_data['type']  # text, image, code, etc.
        }).execute()

    def get_ecosystem_health(self):
        """Get overall ecosystem health status"""
        # Aggregate health metrics from all components
        machine_brain_health = self.get_component_health('machine_brain')
        ai_generative_health = self.get_component_health('ai_generative')
        analytics_health = self.get_component_health('analytics')

        overall_health = (
            machine_brain_health * 0.4 +
            ai_generative_health * 0.4 +
            analytics_health * 0.2
        )

        return {
            'overall_health': overall_health,
            'components': {
                'machine_brain': machine_brain_health,
                'ai_generative': ai_generative_health,
                'analytics': analytics_health
            },
            'status': self.get_health_status(overall_health)
        }
```

Revenue Analytics & Optimization

Revenue Trends



Revenue Sources



Monthly Revenue
$45,230
+23.4% from last month

AI Services
$28,150
62.3% of total revenue

Automation
$12,480
27.6% of total revenue

Analytics
$4,600
10.1% of total revenue

Revenue Optimization Strategy

```python
# Revenue Optimization Engine
class RevenueOptimizer:
    def __init__(self):
        self.supabase = create_client(
            os.getenv('SUPABASE_URL'),
            os.getenv('SUPABASE_KEY')
        )

    def analyze_revenue_patterns(self):
        """Analyze revenue patterns and identify optimization opportunities"""
        # Get revenue data for analysis
        response = self.supabase.table('revenue_analytics').select('*').gte(
            'date', (datetime.now() - timedelta(days=90)).date().isoformat()
        ).execute()

        df = pd.DataFrame(response.data)

        # Analyze by revenue source
        source_performance = df.groupby('revenue_source').agg({
            'amount': ['sum', 'mean', 'count'],
            'conversion_rate': 'mean'
        }).round(2)

        # Identify top performing sources
        top_sources = source_performance.sort_values(
            ('amount', 'sum'), ascending=False
        ).head(5)

        # Generate optimization recommendations
        recommendations = self.generate_recommendations(source_performance)

        return {
            'top_sources': top_sources.to_dict(),
            'recommendations': recommendations,
            'total_revenue': df['amount'].sum(),
            'growth_rate': self.calculate_growth_rate(df)
        }

    def generate_recommendations(self, performance_data):
        """Generate AI-powered revenue optimization recommendations"""
        recommendations = []

        # Example recommendation logic
        for source, data in performance_data.iterrows():
            avg_amount = data[('amount', 'mean')]
            conversion_rate = data[('conversion_rate', 'mean')]

            if conversion_rate < 0.05:  # Less than 5% conversion
                recommendations.append({
                    'source': source,
                    'type': 'conversion_optimization',
                    'priority': 'high',
                    'suggestion': f'Optimize {source} conversion funnel - current rate {conversion_rat
                })

            if avg_amount < 100:  # Low average revenue
                recommendations.append({
                    'source': source,
                    'type': 'pricing_optimization',
                    'priority': 'medium',
                    'suggestion': f'Consider premium pricing for {source} - current avg ${avg_amount:
                })

        return recommendations

    def predict_revenue_targets(self):
        """Predict achievable revenue targets based on current trends"""
        # Fetch recent performance data
        response = self.supabase.table('revenue_analytics').select('*').gte(
            'date', (datetime.now() - timedelta(days=30)).date().isoformat()
        ).execute()

        df = pd.DataFrame(response.data)
        daily_revenue = df.groupby('date')['amount'].sum()

        # Calculate trend and seasonality
        current_avg = daily_revenue.mean()
        growth_trend = self.calculate_trend(daily_revenue.values)

        # Predict targets
        targets = {
            'daily_target': current_avg * 1.15,  # 15% improvement
            'weekly_target': current_avg * 7 * 1.15,
            'monthly_target': current_avg * 30 * 1.15,
            'confidence': 0.78
        }

        return targets
```

Implementation Roadmap

1 Phase 1: Foundation Setup (Week 1-2)

- Setup Supabase project dan database schema
- Configure GitHub Actions workflows
- Implement basic metrics collection
- Create initial dashboard structure

2 Phase 2: Real-time Analytics (Week 3-4)

- Implement real-time subscriptions
- Create comprehensive monitoring dashboard
- Setup automated alert systems
- Integration dengan Machine Brain component

3 Phase 3: Predictive Analytics (Week 5-6)

- Implement machine learning models
- Create predictive dashboards
- Revenue optimization engine
- Integration dengan AI Generative component

4 Phase 4: Advanced Features (Week 7-8)

- Quantum-inspired analytics algorithms
- Advanced business intelligence
- Automated optimization recommendations
- Full ecosystem integration testing

Security & Best Practices

Security Measures

- ✓ Row Level Security (RLS) pada semua Supabase tables
- ✓ Encrypted environment variables untuk API keys
- ✓ Rate limiting untuk API endpoints
- ✓ Data anonymization untuk sensitive metrics

Performance Optimization

- ✓ Database indexing untuk query optimization
- ✓ Caching layer untuk frequent queries
- ✓ Batch processing untuk large datasets
- ✓ Asynchronous operations untuk real-time updates

Ready for The New Civilization

AI Analytics component siap untuk mengukur, menganalisis, dan mengoptimalkan seluruh ekosistem menuju revolusi peradaban baru.

Real-time Monitoring
Comprehensive visibility across all ecosystem components

Predictive Intelligence
AI-powered forecasting dan optimization recommendations

Revenue Growth
Data-driven strategies untuk mencapai target $50K+/month

Made with Genspark